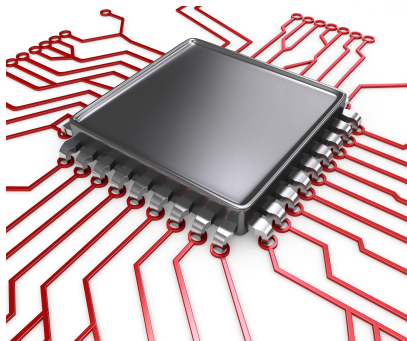




Embedded Systems

Microcontroller Work

H. Cassé
hugues.casse@irit.fr

2022

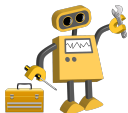
Bare Metal

2 approaches to embedded programming:

- ▶ Linux – requires lot of hardware resources
- ▶ bare metal +/- OS as a library

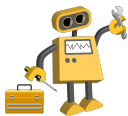
What is the difference with usual programming?

- ▶ heterogeneous hardware
- ▶ set of physical requirements
- ▶ types of sensors and actuators
- ▶ types of applications



Outline

- 1 Introduction
- 2 Requirements
- 3 Programming Input/Output
- 4 Programming with Interrupts
- 5 Conclusion



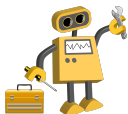
Requirements

Server/desktop/laptop:

- ▶ safety is better
- ▶ expensive to very expensive
- ▶ lowly robust

Embedded system:

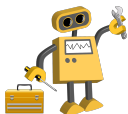
- ▶ safety (often stringent requirement)
- ▶ often cheap
- ▶ power consumption
- ▶ often very robust



Hardware & Software

Server/desktop/laptop:

- ▶ AMD/Intel 64b μ P
(2-3GHz)
x86/ARM
- ▶ 2-4 Gb DRAM
- ▶ powerful OS (Linux, MacOSX, Windows)
- ▶ Ethernet / wifi
- ▶ keyboard, mouse, screen



Embedded system:

- ▶ 8/16/32-bit μ P
(16MHz-1GHz)
ARM, TriCore, Leon,
AVR, etc
- ▶ 20Kb-100Mb SRAM,
1Mb-16MB flash
- ▶ bare-metal / OS as a
library
- ▶ Ethernet, wifi, wireless,
Xbee, etc, none
- ▶ LED, button, specific
sensors and actuators

Microcontroller

Chip grouping:

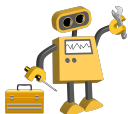
- ▶ execution core(s)
- ▶ memory
- ▶ driver for devices

SOC (*System On Chip*):

- ▶ several cores
- ▶ several memories
- ▶ interconnect network

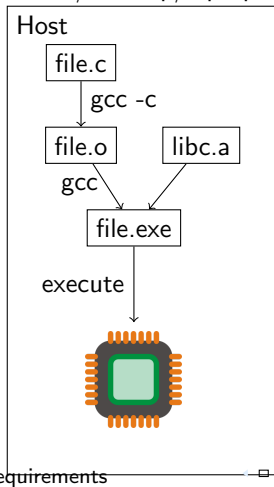
Example: STM32F407
(STMicro)

- ▶ ARM Cortex M4+FPU (monocore)
- ▶ SRAM (main 112Kb, 16Kb)
- ▶ flash (1Mb)
- ▶ network: Ethernet, USB, I2C, USART, SPI
- ▶ peripherals: GPIO, timers, ADC, DAC

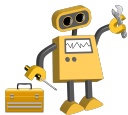
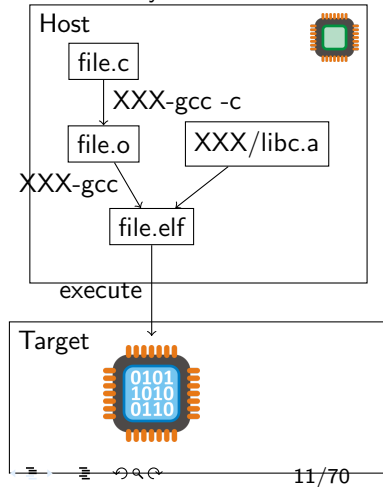


Programming

Server/desktop/laptop:

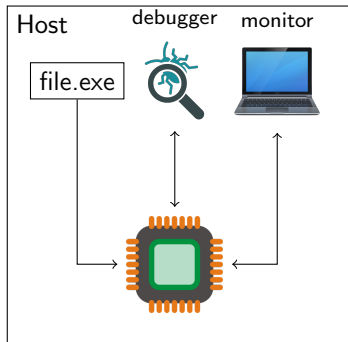


Embedded System:

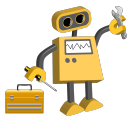
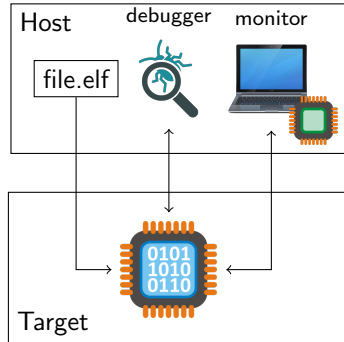


Debugging & Testing

Server/desktop/laptop:



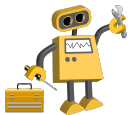
Embedded System:



Host/Target links: USART, USB, JTag

Outline

- 1 Introduction
- 2 Requirements
- 3 Programming Input/Output
- 4 Programming with Interrupts
- 5 Conclusion



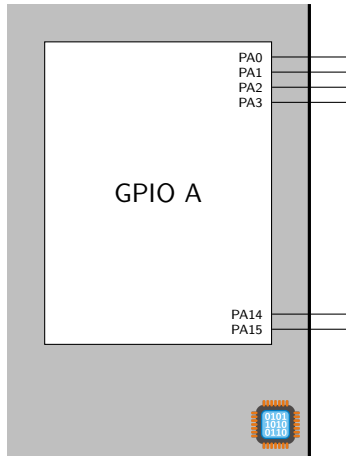
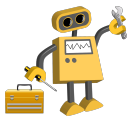
Example: GPIO

Generic Parallel Input/Output

- ▶ electric lines (*pins*)
- ▶ bidirectional
- ▶ output 0V or 3.3V
- ▶ input 0V to 5V
- ▶ any sensor producing pulses (width, frequency)
- ▶ any actuator supporting *all-or-none* signal
- ▶ in fact: very used

STM32F407

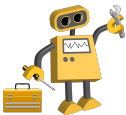
- ▶ GPIO A-K (11 GPIO)
- ▶ 16 pins/GPIO



How to program?

GPIO drivers controlled by registers mapped in memory:

- ▶ `GPIO_MODER` – configure input/output
- ▶ `GPIO_IDR` – value as input
- ▶ `GPIO_ODR` – value as output
- ▶ `GPIO_BSRR` – set/reset output



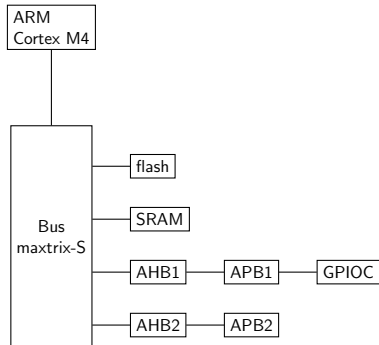
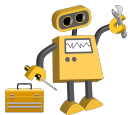
How to program?

GPIO drivers controlled by registers mapped in memory:

- ▶ GPIO_MODER – configure input/output
- ▶ GPIO_IDR – value as input
- ▶ GPIO_ODR – value as output
- ▶ GPIO_BSRR – set/reset output

address = driver address + offset

- ▶ GPIO_IDR offset = 0x10
- ▶ GPIO C base = 0x40020800
- ▶ address = 0x40020810



AHB = Amba High-speed Bus

APB = Amba Peripheral Bus

Finding the base addresses?

From the microcontroller reference manual:

RM0090 - Reference manual

*STM32F405/415, STM32F407/417, STM32F427/437 and
STM32F429/439 advanced Arm [®]-based 32-bit MCUs*

In memory map section:

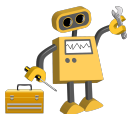
0x4002 3800 - 0x4002 3BFF	RCC	AHB1
0x4002 3000 - 0x4002 33FF	CRC	
0x4002 2800 - 0x4002 2BFF	GPIOK	
0x4002 2400 - 0x4002 27FF	GPIOJ	
0x4002 2000 - 0x4002 23FF	GPIOI	
0x4002 1C00 - 0x4002 1FFF	GPIOH	
0x4002 1800 - 0x4002 1BFF	GPIOG	
0x4002 1400 - 0x4002 17FF	GPIOF	
0x4002 1000 - 0x4002 13FF	GPIOE	

[Section 7.3.24: RCC register map on page 265](#)

[Section 4.4.4: CRC register map on page 115](#)

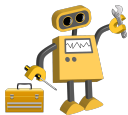
[Section 8.4.11: GPIO register map on page 287](#)

[Section 8.4.11: GPIO register map on page 287](#)



Use of defines:

```
#define GPIO_IDR    0x10
#define GPIO_ODR    0x14
...
#define GPIOC_BASE  0x40020800
...
```

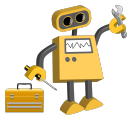


Use of defines:

```
#define GPIO_IDR    0x10
#define GPIO_ODR    0x14
...
#define GPIOC_BASE  0x40020800
...
```

Define to access IDR of GPIOC:

```
#define GPIOC_IDR    \
    (*(volatile uint32_t *) (GPIOC_BASE + GPIO_IDR))
```



Use of defines:

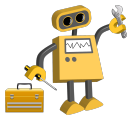
```
#define GPIO_IDR    0x10
#define GPIO_ODR    0x14
...
#define GPIOC_BASE  0x40020800
...
```

Define to access IDR of GPIOC:

```
#define GPIOC_IDR    \
    (*(volatile uint32_t *) (GPIOC_BASE + GPIO_IDR))
```

Used with:

```
GPIOC_IDR = 0;
if (GPIOC_IDR == 0b0100) ...
```



Use of defines:

```
#define GPIO_IDR    0x10
#define GPIO_ODR    0x14
...
#define GPIOC_BASE  0x40020800
...
```

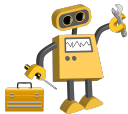
Define to access IDR of GPIOC:

```
#define GPIOC_IDR    \
    *((volatile uint32_t *) (GPIOC_BASE + GPIO_IDR))
```

Used with:

```
GPIOC_IDR = 0;
if(GPIOC_IDR == 0b0100) ...
```

volatile $\Rightarrow$ prevent compiler optimizations

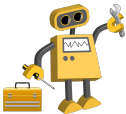


GPIOx_ODR

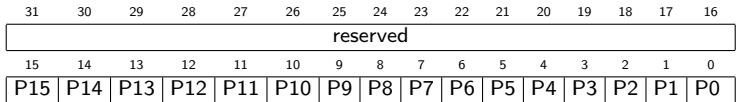
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P15	P14	P13	P12	P11	P10	P9	P8	P7	P6	P5	P4	P3	P2	P1	P0

16 pins $\iff$ 1 bit / pin

- ▶ bit = 0 $\implies$ pin at 0V
- ▶ bit = 1 $\implies$ pin at 5V



GPIOx_ODR

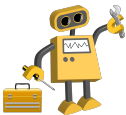


16 pins $\iff$ 1 bit / pin

- ▶ bit = 0 $\implies$ pin at 0V
- ▶ bit = 1 $\implies$ pin at 5V

How to change it?

- 1 read the word
- 2 changes the bits
- 3 write the word



How to change bits?

Use bit-to-bit operations of C:

- ▶ ~ – bit level NOT

$$X = \begin{array}{|c|c|c|c|c|c|} \hline X_{31} & X_{30} & \dots & X_2 & X_1 & X_0 \\ \hline \end{array}$$

$$\sim X = \begin{array}{|c|c|c|c|c|c|} \hline \overline{X_{31}} & \overline{X_{30}} & \dots & \overline{X_2} & \overline{X_1} & \overline{X_0} \\ \hline \end{array}$$

- ▶ & – bit level AND

$$A = \begin{array}{|c|c|c|c|c|c|} \hline A_{31} & A_{30} & \dots & A_2 & A_1 & A_0 \\ \hline \end{array}$$

$$B = \begin{array}{|c|c|c|c|c|c|} \hline B_{31} & B_{30} & \dots & B_2 & B_1 & B_0 \\ \hline \end{array}$$

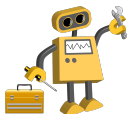
$$A \& B = \begin{array}{|c|c|c|c|c|c|} \hline A_{31} \cdot B_{31} & A_{30} \cdot B_{30} & \dots & A_2 \cdot B_2 & A_1 \cdot B_1 & A_0 \cdot B_0 \\ \hline \end{array}$$

- ▶ | – bit level OR

$$A = \begin{array}{|c|c|c|c|c|c|} \hline A_{31} & A_{30} & \dots & A_2 & A_1 & A_0 \\ \hline \end{array}$$

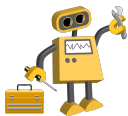
$$B = \begin{array}{|c|c|c|c|c|c|} \hline B_{31} & B_{30} & \dots & B_2 & B_1 & B_0 \\ \hline \end{array}$$

$$A | B = \begin{array}{|c|c|c|c|c|c|} \hline A_{31} + B_{31} & A_{30} + B_{30} & \dots & A_2 + B_2 & A_1 + B_1 & A_0 + B_0 \\ \hline \end{array}$$



Setting one bit to 1

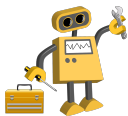
- ▶ 1 is absorbing for +: $A + 1 = 1$
- ▶ 0 is neutral for +: $A + 0 = A$



Setting one bit to 1

- ▶ 1 is absorbing for +: $A + 1 = 1$
- ▶ 0 is neutral for +: $A + 0 = A$

$X =$	X_{31}	X_{30}	...	X_i	...	X_1	X_0
$M =$	0	0	...	1	...	0	0
$X M =$	X_{31}	X_{30}	...	1	...	X_1	X_0



Setting one bit to 1

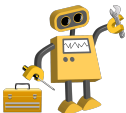
- ▶ 1 is absorbing for +: $A + 1 = 1$
- ▶ 0 is neutral for +: $A + 0 = A$

$X =$	X_{31}	X_{30}	...	X_i	...	X_1	X_0
$M =$	0	0	...	1	...	0	0
$X M =$	X_{31}	X_{30}	...	1	...	X_1	X_0

How to get the mask M where only bit i is 1:

$$M = 1 \ll i$$

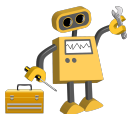
$\ll$ bit shift left,
 $\gg$ bit shift right.



Setting one bit to 0

- ▶ 1 is neutral for $\therefore A.1 = A$
- ▶ 0 is absorbing for $\therefore A.0 = 0$

$X =$	X_{31}	X_{30}	...	X_i	...	X_1	X_0
$M =$	1	1	...	0	...	1	1
$X \& M =$	X_{31}	X_{30}	...	0	...	X_1	X_0



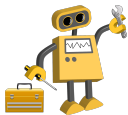
Setting one bit to 0

- ▶ 1 is neutral for $\therefore A.1 = A$
- ▶ 0 is absorbing for $\therefore A.0 = 0$

$X =$	X_{31}	X_{30}	...	X_i	...	X_1	X_0
$M =$	1	1	...	0	...	1	1
$X \& M =$	X_{31}	X_{30}	...	0	...	X_1	X_0

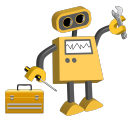
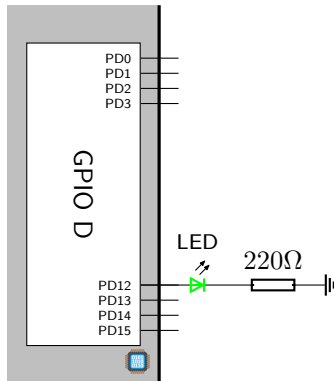
How to get the mask M where only bit i is 0 (and other are 1s):

$$M = \sim (1 \ll i)$$



Controlling a LED

```
#define GREEN_LED 12
```

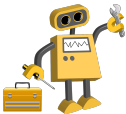
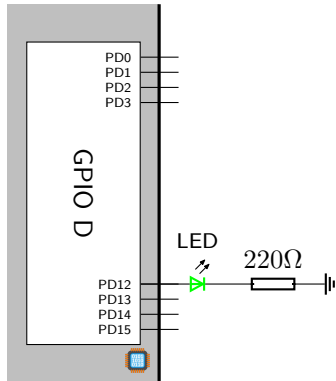


Controlling a LED

```
#define GREEN_LED 12
```

Lighting on:

```
GPIOD_ODR = GPIOD_ODR | (1 << GREEN_LED);
```



Controlling a LED

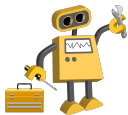
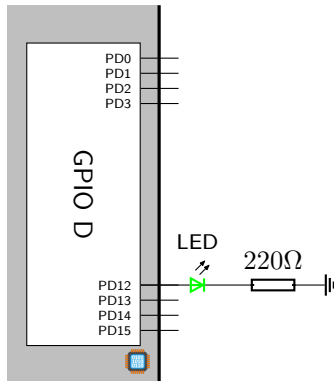
```
#define GREEN_LED 12
```

Lighting on:

```
GPIO_ODR = GPIO_ODR | (1 << GREEN_LED);
```

Lighting off:

```
GPIO_ODR = GPIO_ODR & ~(1 << GREEN_LED);
```



Controlling a LED

```
#define GREEN_LED 12
```

Lighting on:

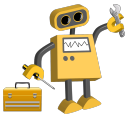
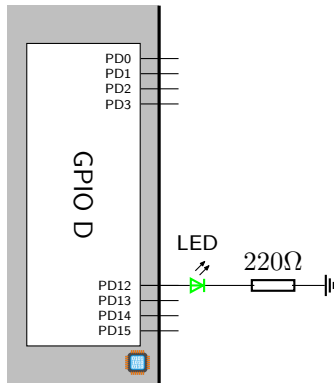
```
GPIO_ODR = GPIO_ODR | (1 << GREEN_LED);
```

Lighting off:

```
GPIO_ODR = GPIO_ODR & ~(1 << GREEN_LED);
```

Exercise: write code to

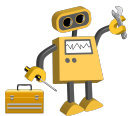
- ▶ switch on LED orange and red (PD13 and PD14)
- ▶ switch off LED green and blue (PD12 and PD15)



Different ways to set the output of GPIO

Register GPIOx_BSRR (Bit Set/Reset)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

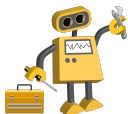


Different ways to set the output of GPIO

Register GPIOx_BSRR (Bit Set/Reset)

[illegible]

- ▶ only 1 are effective (0 ignored)!
- ▶ 1 in bit $i \in [15 - 0] \implies$ set 1 to output pin i
- ▶ 1 in bit $i \in [31 - 16] \implies$ set 0 to output pin $i - 16$

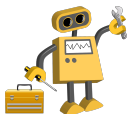


Different ways to set the output of GPIO

Register GPIOx_BSRR (Bit Set/Reset)

[illegible]

- ▶ only 1 are effective (0 ignored)!
- ▶ 1 in bit $i \in [15 - 0] \implies$ set 1 to output pin i
- ▶ 1 in bit $i \in [31 - 16] \implies$ set 0 to output pin $i - 16$
- ▶ switch on green LED:
`GPIOD_BSRR = 1 << GREEN_LED;`

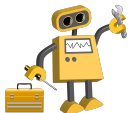


Different ways to set the output of GPIO

Register GPIOx_BSRR (Bit Set/Reset)

[illegible]

- ▶ only 1 are effective (0 ignored)!
- ▶ 1 in bit $i \in [15 - 0] \implies$ set 1 to output pin i
- ▶ 1 in bit $i \in [31 - 16] \implies$ set 0 to output pin $i - 16$
- ▶ switch on green LED:
`GPIOD_BSRR = 1 << GREEN_LED;`
- ▶ switch off green LED:
`GPIOD_BSRR = 1 << (GREEN_LED + 16);`



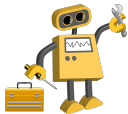
Different ways to set the output of GPIO

Register GPIOx_BSRR (Bit Set/Reset)

[illegible]

- ▶ only 1 are effective (0 ignored)!
- ▶ 1 in bit $i \in [15 - 0] \implies$ set 1 to output pin i
- ▶ 1 in bit $i \in [31 - 16] \implies$ set 0 to output pin $i - 16$
- ▶ switch on green LED:
`GPIOD_BSRR = 1 << GREEN_LED;`
- ▶ switch off green LED:
`GPIOD_BSRR = 1 << (GREEN_LED + 16);`

Exercise: do the same as previous exercise with `GPIOx_BSSR`.



Blinking program

Write a program that blinks the red LED with a period of 1s:

```
#include <stm32/gpio.h>
...
#define RED_LED      14

int main() {
    int HALF_PERIOD = 1000000;

    /* GPIO initialization */
    ...

    /* endless loop */
    while(1) {

        /* wait loop */
        for(int i = 0; i < HALF_PERIOD; i++) NOP;

        /* switch on/off the LED */
        if((GPIO_ODR & (1 << RED_LED)) == 0)
            GPIO_BSRR = 1 << RED_LED;
        else
            GPIO_BSRR = 1 << (RED_LED + 16);
    }
}
```

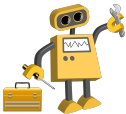
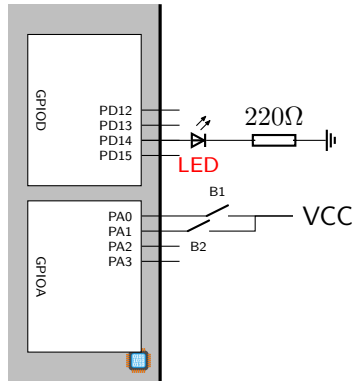


Setting blinking speed

GPIOx_IDR (Input Data Register)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

- ▶ bit i is 0 – input voltage 0V
- ▶ bit i is 1 – input voltage is $\sim 5V$



Setting blinking speed

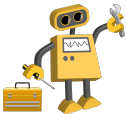
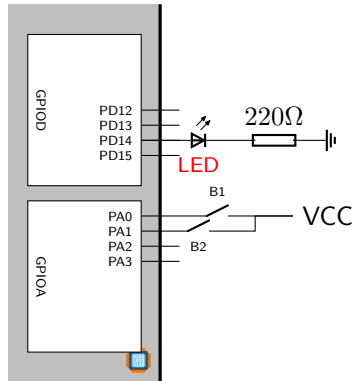
GPIOx_IDR (Input Data Register)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

- ▶ bit i is 0 – input voltage 0V
- ▶ bit i is 1 – input voltage is $\sim 5V$

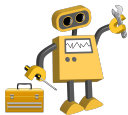
Example

```
#define B1 0
#define B2 1
if((GPIOA.IDR & (1 << B1)) != 0)
    /* pin 0 = 1, B1 is pushed */
else
    /* pin 0 = 0, B1 is released */
```



How to test a bit value?

Problem: inside a value x , is bit i 0 or 1?

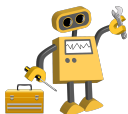


How to test a bit value?

Problem: inside a value x , is bit i 0 or 1?

	31						0
x	X_{31}	X_{30}	...	X_i	...	X_1	X_0
M	0	0	...	1	...	0	0
$x \& M$	0	0	...	X_i	...	0	0

$= R$



How to test a bit value?

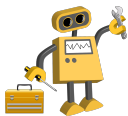
Problem: inside a value x , is bit i 0 or 1?

	31						0
x	X_{31}	X_{30}	...	X_i	...	X_1	X_0
M	0	0	...	1	...	0	0
$x \& M$	0	0	...	X_i	...	0	0

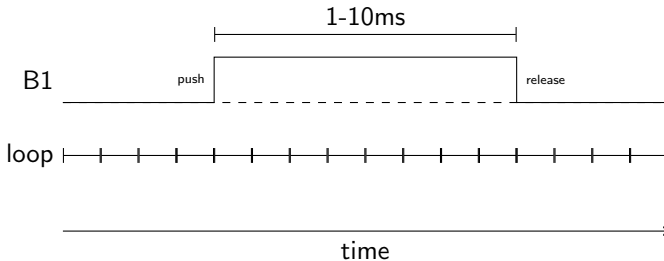
$= R$

let $R = 0\ 0\ \dots\ X_i\ \dots\ 0\ 0$

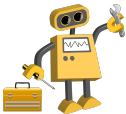
- ▶ if $R \& M = 0$ then $X_i = 0$,
- ▶ if $R \& M \neq 0$ then $X_i = 1$.



What is a click?



```
int b1_state = 0;
if((GPIOA_IDR & (1 << B1)) != 0)
    b1_state = 1;          /* button pushed */
else if(b1_state == 1) {  /* assume (GPIOA_IDR & (1 << B1)) == 0 */
    b1_state = 0;          /* button released after push: click */
    /* action here */
}
```



Blinker with speed up/down buttons

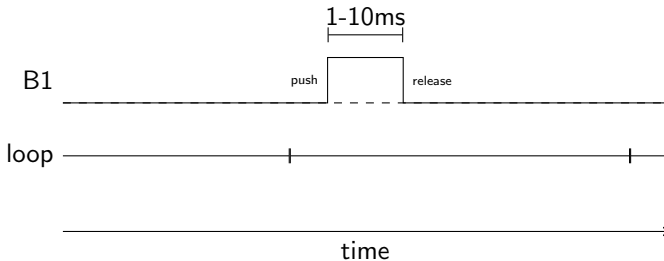
```
int b1_state = 0, b2_state = 0;
while(1) {
    for(int i = 0; i < HALF_PERIOD; i++);

    /* invert LED */
    if((GPIO_ODR & (1 << RED_LED)) == 0)
        GPIO_BSRR = 1 << RED_LED;
    else
        GPIO_BSRR = 1 << (RED_LED + 16);

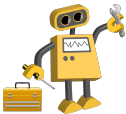
    /* poll buttons */
    if((GPIO_IDR & (1 << B1)) != 0)
        b1_state = 1;
    else if(b1_state) {
        b1_state = 0;
        HALF_PERIOD += 1000;
    }
    if((GPIO_IDR & (1 << B2)) != 0)
        b2_state = 1;
    else if(b2_state) {
        b2_state = 0;
        HALF_PERIOD -= 1000;
    }
}
```



What will happen?



- ▶ button polling must be put in waiting loop
- ▶ additional processing in wait loop $\Rightarrow$ count reduction



Better blinker version

```
int HALF_PERIOD = 200000;
int b1_state = 0, b2_state = 0;
while(1) {

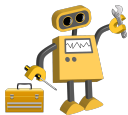
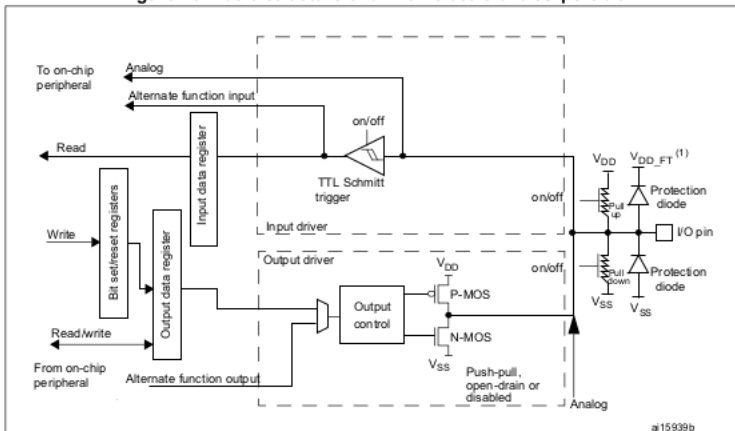
    /* waiting loop and button test */
    for(int i = 0; i < HALF_PERIOD; i++) {
        if((GPIO_IDR & (1 << B1)) != 0)
            b1_state = 1;
        else if(b1_state) {
            b1_state = 0;
            HALF_PERIOD += 200;
        }
        if((GPIO_IDR & (1 << B2)) != 0)
            b2_state = 1;
        else if(b2_state) {
            b2_state = 0;
            HALF_PERIOD -= 200;
        }
    }

    /* invert LED */
    if((GPIO_ODR & (1 << RED_LED)) == 0)
        GPIO_BSRR = 1 << RED_LED;
    else
        GPIO_BSRR = 1 << (RED_LED << 16);
}
```



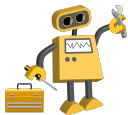
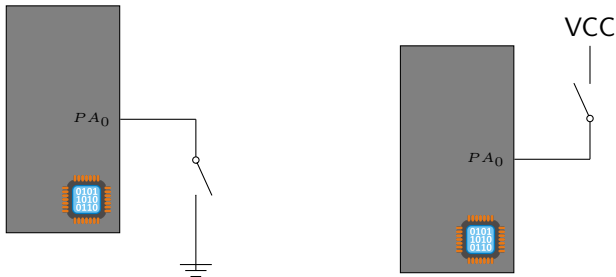
About initialization: GPIO circuit

Figure 25. Basic structure of a five-volt tolerant I/O port bit



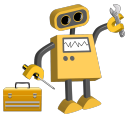
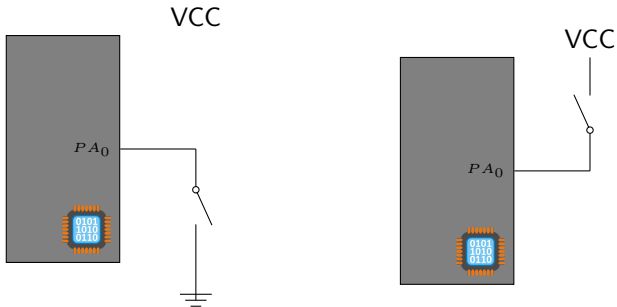
About pull-up/down (input)

Avoid short-circuits/no-floating input



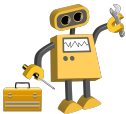
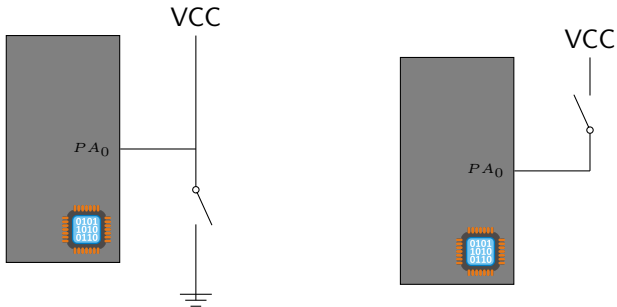
About pull-up/down (input)

Avoid short-circuits/no-floating input



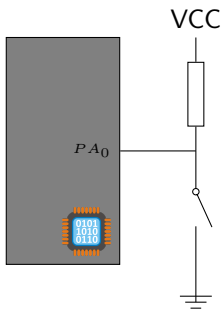
About pull-up/down (input)

Avoid short-circuits/no-floating input

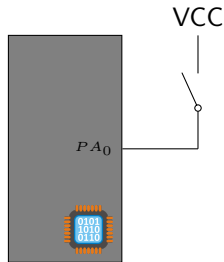


About pull-up/down (input)

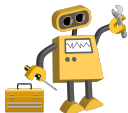
Avoid short-circuits/no-floating input



pull-up resistor

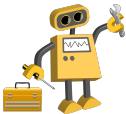
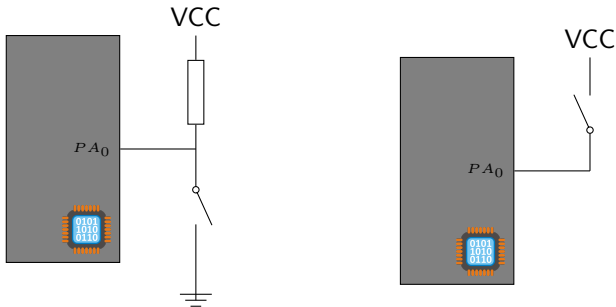


pull-down resistor



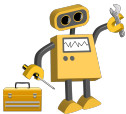
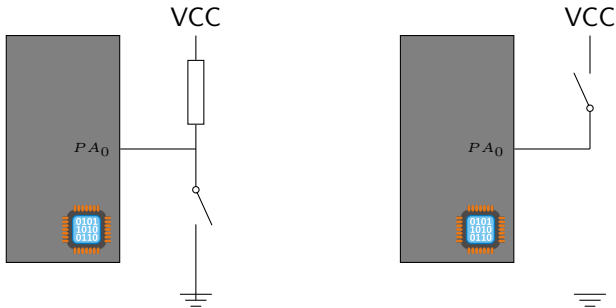
About pull-up/down (input)

Avoid short-circuits/no-floating input



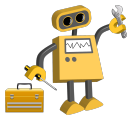
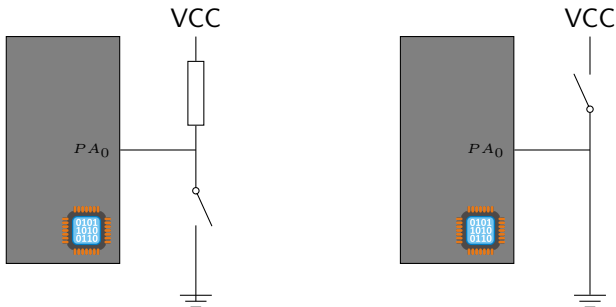
About pull-up/down (input)

Avoid short-circuits/no-floating input



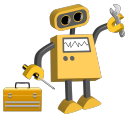
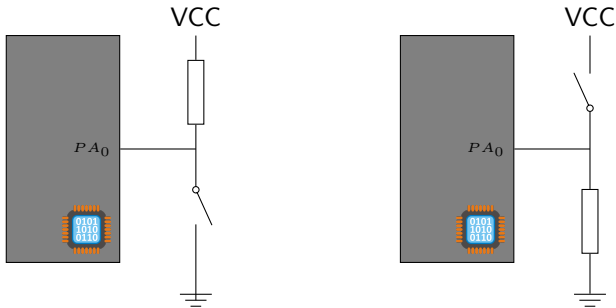
About pull-up/down (input)

Avoid short-circuits/no-floating input

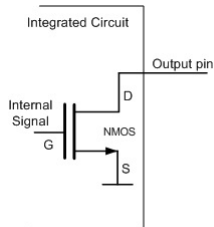
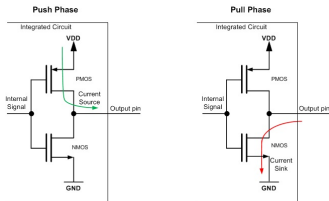


About pull-up/down (input)

Avoid short-circuits/no-floating input

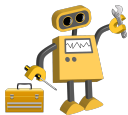


About push-pull/open-drain



- ▶ never floating
- ▶ point-to-point connection
- ▶ NPN transistor open
⇒ 1
- ▶ PNP transistor open
⇒ 0

- ▶ completed by pull-up resistor
- ▶ support bus connection
- ▶ all transistors closed $\Rightarrow 1$
- ▶ PNP transistor open $\Rightarrow 0$



GPIOx_MODER

MODE Register

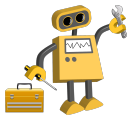
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
P15	P14	P13	P12	P11	P10	P9	P8								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P7	P6	P5	P4	P3	P2	P1	P0								

`GPIO_MODER_IN` (00) – input mode (reset state)

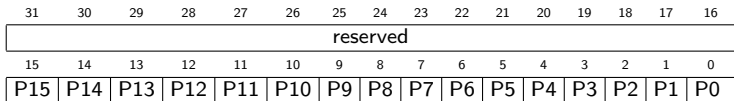
`GPIO_MODER_OUT` (01) – output mode

`GPIO_MODER_ALT` (10) – alternate function mode (seen later)

`GPIO_MODER_ANA` (11) – analog mode (seen later)

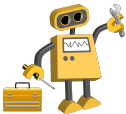


Output OTYPER Register



0 = push-pull (reset state)

1 = open-drain



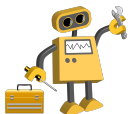
Pull-Up/Pull-Down Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
P15	P14	P13	P12	P11	P10	P9	P8								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P7	P6	P5	P4	P3	P2	P1	P0								

`GPIO_PUPDR_NO` (00) – no pull-up, no pull-down

`GPIO_PUPDR_PU` (01) – pull-up

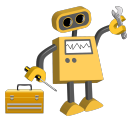
`GPIO_PUPDR_PD` (10) – pull-down



(a) How to set up a range of bits?

Problem: setting value x starting at bit i on l bits in word y on l bits.

Example: setting 101 into 0011 0001 at position 3.
(should result in 0010 1001)



(a) How to set up a range of bits?

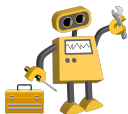
Problem: setting value x starting at bit i on l bits in word y on l bits.

Example: setting 101 into 0011 0001 at position 3.

(should result in 0010 1001)

Step 1: set the ones – $y_2 = y|(x \ll i)$

$$\begin{aligned} & 0011\ 0001 \mid (0000\ 0101 \ll 3) \\ = & 0011\ 0001 \mid (00\ 10\ 1000) \\ = & 0011\ 1001 \end{aligned}$$



(a) How to set up a range of bits?

Problem: setting value x starting at bit i on l bits in word y on l bits.

Example: setting 101 into 0011 0001 at position 3.

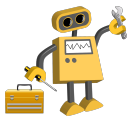
(should result in 0010 1001)

Step 1: set the ones – $y_2 = y | (x \ll i)$

$$\begin{aligned} & 0011\ 0001 \mid (0000\ 0101 \ll 3) \\ &= 0011\ 0001 \mid (00\ 10\ 1000) \\ &= 0011\ 1001 \end{aligned}$$

Step 2: set the zeroes – $y_2 = y \& \sim(x \ll i)$

$$\begin{aligned} & 0011\ 1001 \& \sim(0000\ 0101 \ll 3) \\ &= 0011\ 1001 \& \sim(0010\ 1000) \\ &= 0011\ 1001 \& 1101\ 0111 \\ &= 0001\ 0001 \end{aligned}$$



(a) How to set up a range of bits?

Problem: setting value x starting at bit i on l bits in word y on l bits.

Example: setting 101 into 0011 0001 at position 3.

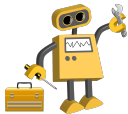
(should result in 0010 1001)

Step 1: set the ones – $y_2 = y | (x \ll i)$

$$\begin{aligned} & 0011\ 0001 \mid (0000\ 0101 \ll 3) \\ &= 0011\ 0001 \mid (00\ 10\ 1000) \\ &= 0011\ 1001 \end{aligned}$$

Step 2: set the zeroes – $y_2 = y \ \& \ (x \ll i)$

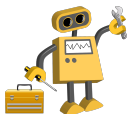
$$\begin{aligned} & 0011\ 1001 \ \& \ \sim(0000\ 0101 \ll 3) \\ &= 0011\ 1001 \ \& \ 0010\ 1000 \\ &= 0010\ 1000 \end{aligned}$$



(b) How to set up a range of bits?

Step 1: reset the range – $y_2 = y \& \sim(MASK(l) \ll i)$

$$\begin{aligned} & 0011\ 0001 \& \sim(0000\ 0111 \ll 3) \\ = & 0011\ 0001 \& \sim(00\ 11\ 1000) \\ = & 0011\ 0001 \& \sim(11\ 00\ 0111) \\ = & 0000\ 0001 \end{aligned}$$



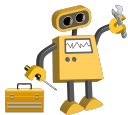
(b) How to set up a range of bits?

Step 1: reset the range – $y_2 = y \ \& \ \sim(MASK(l) \ll i)$

$$\begin{aligned} & 0011\ 0001 \ \& \ \sim(0000\ 0111 \ll 3) \\ &= 0011\ 0001 \ \& \ \sim(00\ 11\ 1000) \\ &= 0011\ 0001 \ \& \ \sim(11\ 00\ 0111) \\ &= 0000\ 0001 \end{aligned}$$

Step 2: set the ones – $y_3 = y_2 | (x \ll i)$

$$\begin{aligned} & 0000\ 0001 \ | \ (0000\ 0101 \ll 3) \\ &= 0000\ 0001 \ | \ (00\ 10\ 1000) \\ &= 0010\ 1001 \end{aligned}$$



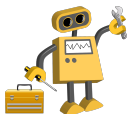
(b) How to set up a range of bits?

Step 1: reset the range – $y_2 = y \& \sim(MASK(l) \ll i)$

$$\begin{aligned} & 0011\ 0001 \& \sim(0000\ 0111 \ll 3) \\ = & 0011\ 0001 \& \sim(00\ 11\ 1000) \\ = & 0011\ 0001 \& \sim(11\ 00\ 0111) \\ = & 0000\ 0001 \end{aligned}$$

Step 2: set the ones – $y_3 = y_2 | (x \ll i)$

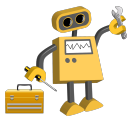
$$\begin{aligned} & 0000\ 0001 | (0000\ 0101 \ll 3) \\ = & 0000\ 0001 | (00\ 10\ 1000) \\ = & 0010\ 1001 \end{aligned}$$



```
#define MASK(l) ((1 << (l))-1)
#define REP_BITS(y, i, l, x) (((y) & (~MASK(l) << (i))) | ((x) << (i)))
```

How get a range of bits?

Problem: how get bit range starting at i and expanding on l bits from x ?

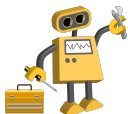


How get a range of bits?

Problem: how get bit range starting at i and expanding on l bits from x ?

Solution: $(x \gg i) \& ((1 \ll l)-1)$

Example: $x = 01101000$, $i = 2$, $l = 2$, expected 00000010



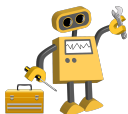
How get a range of bits?

Problem: how get bit range starting at i and expanding on l bits from x ?

Solution: $(x \gg i) \& ((1 \ll l)-1)$

Example: $x = 01101000$, $i = 2$, $l = 2$, expected 00000010

	7	6	5	4	3	2	1	0
x	0	1	1	0	1	0	0	0
$x \gg i$	0	0	0	1	1	0	1	0
$M = (1 \ll l)-1$	0	0	0	0	0	0	1	1
$(x \gg i) \& M$	0	0	0	0	0	0	1	0



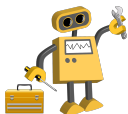
How get a range of bits?

Problem: how get bit range starting at i and expanding on l bits from x ?

Solution: $(x \gg i) \& ((1 \ll l)-1)$

Example: $x = 01101000$, $i = 2$, $l = 2$, expected 00000010

	7	6	5	4	3	2	1	0
x	0	1	1	0	1	0	0	0
$x \gg i$	0	0	0	1	1	0	1	0
$M = (1 \ll l)-1$	0	0	0	0	0	0	1	1
$(x \gg i) \& M$	0	0	0	0	0	0	1	0

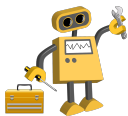


```
#define MASK(l) ((1 << (l))-1)
#define GET_BITS(x, i, l) (((x) >> (i)) & MASK(l))
```

Configuration

Red LED (PD14) programmed as output, push-pull:

```
GPIO_MODER = REP_BITS(GPIO_MODER, RED_LED*2, 2, GPIO_MODER_OUT);  
GPIO_OTYPER = GPIO_OTYPER & ~(1 << RED_LED);
```



Configuration

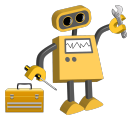
Red LED (PD14) programmed as output, push-pull:

```
GPIOA_MODER = REP_BITS(GPIOA_MODER, RED_LED*2, 2, GPIOA_MODER_OUT);  
GPIOA_OTYPER = GPIOA_OTYPER & ~(1 << RED_LED);
```

B1, B2 input, pull-down (PA0, PA1):

```
GPIOA_MODER = REP_BITS(GPIOA_MODER, B1*2, 2, GPIOA_MODER_IN);  
GPIOA_PUPDR = REP_BITS(GPIOA_PUPDR, B1*2, 2, GPIOA_PUPDR_PD);
```

```
GPIOA_MODER = REP_BITS(GPIOA_MODER, B2*2, 2, GPIOA_MODER_IN);  
GPIOA_PUPDR = REP_BITS(GPIOA_PUPDR, B2*2, 2, GPIOA_PUPDR_PD);
```



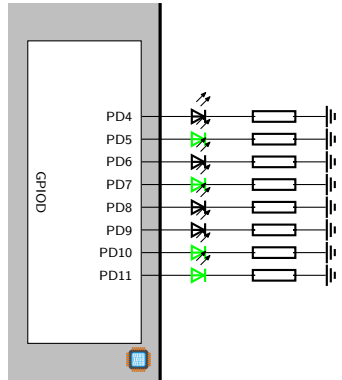
Exercise (1)

GPIO D pins 4 to 11 (8 pins) are connected to LEDs to display a pattern 0b11001010 that we want to shift to the right to achieve a visual effect.

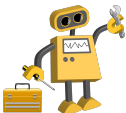
To count time, an empty loop has to iterate 1000000 times.

The current state of the pattern is stored in global variable named `pattern`.

- 1 Initialize the PIO in `init()`.
- 2 Write a function `display()` that displays the pattern on the GPIO.
- 3 Write the `main()` function.



```
char pattern = 0b11001010;  
void display();  
void init();
```



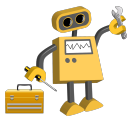
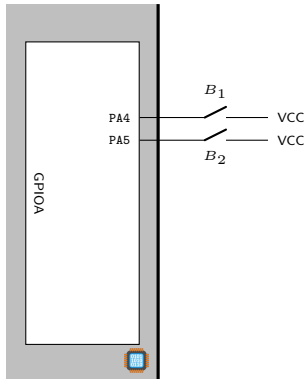
Exercise (2)

Followup of exercise (1).

We consider that pin A4 and A5 are connected to push buttons named, respectively, B_1 and B_2 :

- ▶ Pushing on B_1 make the pattern to shift left.
- ▶ Pushing on B_2 make the pattern to shift to right.

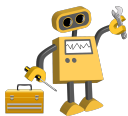
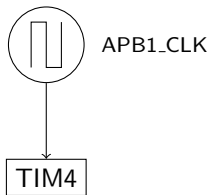
Change the endless loop in `main()` that manages the push buttons to implement this extension.



Better timing with Timer

TIM4: timer counter

- ▶ count events
- ▶ events from APB1_CLK (42MHz) – APB1 clock



Better timing with Timer

TIM4: timer counter

- ▶ count events
- ▶ events from APB1_CLK (42MHz) – APB1 clock

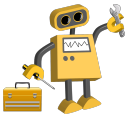
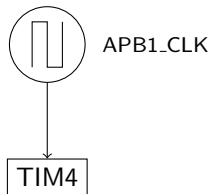
Registers:

`TIM4_CNT` – event CouNTER register

`TIM4_ARR` – Auto-Reload Register

`TIM4_SR` – Status Register

`TIM4_PSC` – PreSCaler register



Better timing with Timer

TIM4: timer counter

- ▶ count events
- ▶ events from APB1_CLK (42MHz) – APB1 clock

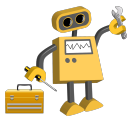
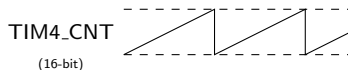
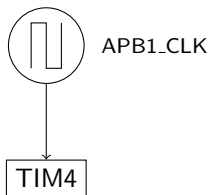
Registers:

TIM4_CNT – event CouNTER register

TIM4_ARR – Auto-Reload Register

TIM4_SR – Status Register

TIM4_PSC – PreSCaler register



TIM4 Configuration

```
#define WAIT_PSC      1000
#define WAIT_DELAY   (APB1_CLK / PSC / 2)
int HALF_PERIOD = WAIT_DELAY;

void init_TIM4() {

    /* stop the timer: TIM_CEN set to 0 */
    TIM4_CR1 = 0;

    /* prescaling: clock = APB1_CLK / 1000 */
    TIM4_PSC = WAIT_PSC-1;

    /* 0.5s = 42MHz / 2 / 1000 */
    TIM4_ARR = HALF_PERIOD;

    /* reset the counter (Update Generation) */
    TIM4_EGR = TIM_UG;

    /* reset the status register */
    TIM4_SR = 0;

    /* Counter ENable */
    TIM4_CR1 = TIM_CEN;
}
```



Blink with TIM4

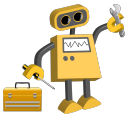
```
TIM4_CR1 = TIM_CEN;
while(1) {

    /* waiting loop and button test */
    while((TIM4_SR & TIM_UIF) == 0) {
        if((GPIO_IDR & (1 << B1)) != 0)
            b1_state = 1;
        else if(b1_state) {
            b1_state = 0;
            HALF_PERIOD += 200;
        }
        if((GPIO_IDR & (1 << B2)) != 0)
            b2_state = 1;
        else if(b2_state) {
            b2_state = 0;
            HALF_PERIOD -= 200;
        }
    }

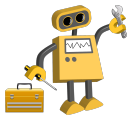
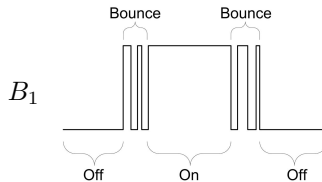
    /* invert LED */
    if((GPIO_ODR & (1 << RED_LED)) == 0)
        GPIO_BSRR = 1 << RED_LED;
    else
        GPIO_BSRR = 1 << (RED_LED << 16);
    TIM4_ARR = HALF_PERIOD;
    TIM4_SR = 0;
}
```



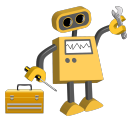
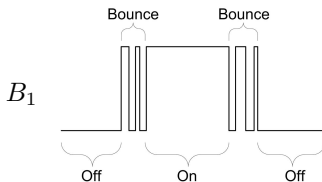
Principle of reality: bouncing



Principle of reality: bouncing



Principle of reality: bouncing



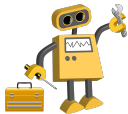
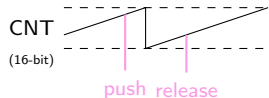
Consider that a normal human user cannot push-release $< 50ms$.

Blink and debounce

```
int last_b1 = 0;
TIM4_CR1 = TIM_CEN;
while(1) {

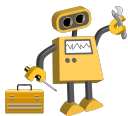
    /* waiting loop and button test */
    while((TIM4_SR & TIM_UIF) == 0) {
        if((GPIO_IDR & (1 << B1)) != 0) {
            if(!b1.state) {
                last_b1 = TIM4_CNT; // record date of push
                b1.state = 1;
            }
            else if(b1.state) {
                int now = TIM4_CNT; // get data of release
                if(now <= last_b1) // timer wrap issue fix
                    now += HALF_PERIOD;
                if(now - last_b1 >= DELAY_50MS) {
                    b1.state = 0;
                    HALF_PERIOD += 200;
                }
            }
        }
        ...
        TIM4_SR = 0;
    }

    /* invert LED */
    ...
}
```



Outline

- 1 Introduction
- 2 Requirements
- 3 Programming Input/Output
- 4 Programming with Interrupts
- 5 Conclusion

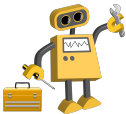


Polling approach

```
while(1) {
    if(test_R1)
        action1();
    if(test_R2)
        action2();
    if(test_R3)
        action3();
    ...
}
```

Pros:

- ▶ easy to write

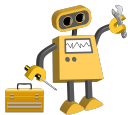


Polling approach

```
while(1) {
    if(test_R1)
        action1();
    if(test_R2)
        action2();
    if(test_R3)
        action3();
    ...
}
```

Pros:

- ▶ easy to write
- ▶ easy to understand

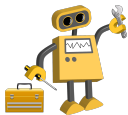


Polling approach

```
while(1) {
    if(test_R1)
        action1();
    if(test_R2)
        action2();
    if(test_R3)
        action3();
    ...
}
```

Pros:

- ▶ easy to write
- ▶ easy to understand
- ▶ easy to debug

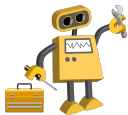


Polling approach

```
while(1) {
    if(test_R1)
        action1();
    if(test_R2)
        action2();
    if(test_R3)
        action3();
    ...
}
```

Pros:

- ▶ easy to write
- ▶ easy to understand
- ▶ easy to debug



Polling approach

```
while(1) {
    if(test_R1)
        action1();
    if(test_R2)
        action2();
    if(test_R3)
        action3();
    ...
}
```

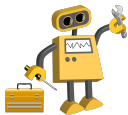
Pros:

- ▶ easy to write
- ▶ easy to understand
- ▶ easy to debug

Cons: more and more tests

 $\Rightarrow$

- ▶ increase action latency



Polling approach

```
while(1) {
    if(test_R1)
        action1();
    if(test_R2)
        action2();
    if(test_R3)
        action3();
    ...
}
```

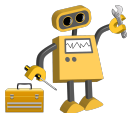
Pros:

- ▶ easy to write
- ▶ easy to understand
- ▶ easy to debug

Cons: more and more tests

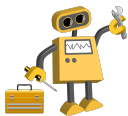
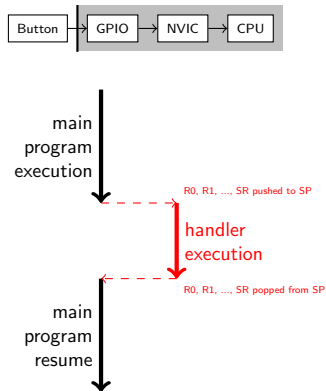
 $\Rightarrow$

- ▶ increase action latency
- ▶ event miss



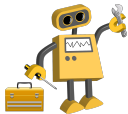
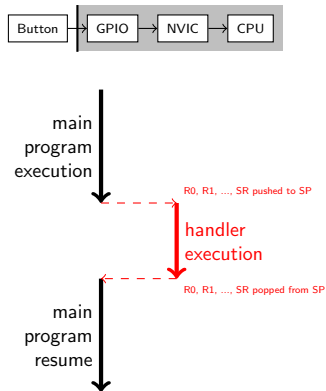
Interrupts: principles

- ▶ main program executes (main loop)



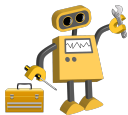
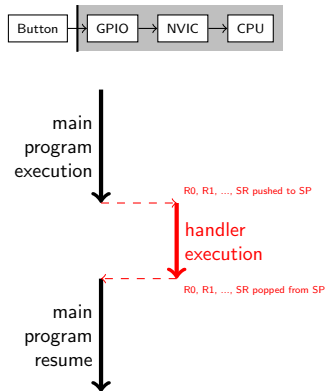
Interrupts: principles

- ▶ main program executes (main loop)
- ▶ change in the input (GPIO button)



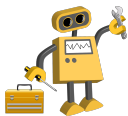
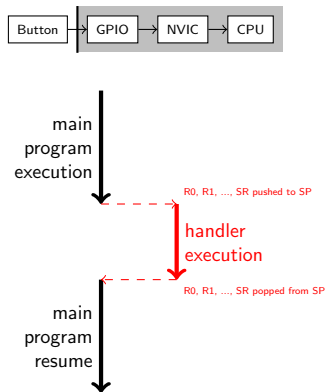
Interrupts: principles

- ▶ main program executes (main loop)
- ▶ change in the input (GPIO button)
- ▶ IRQ signal (Interrupt ReQuest) from GPIO



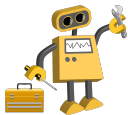
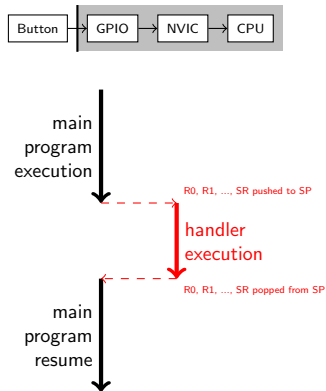
Interrupts: principles

- ▶ main program executes (main loop)
- ▶ change in the input (GPIO button)
- ▶ IRQ signal (Interrupt ReQuest) from GPIO
- ▶ IRQ signal passed to IRQ controller



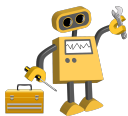
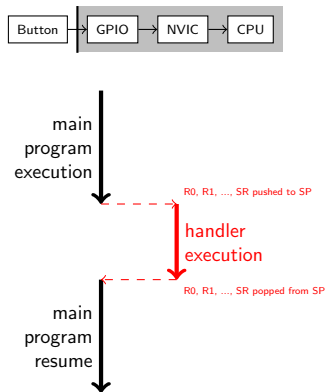
Interrupts: principles

- ▶ main program executes (main loop)
- ▶ change in the input (GPIO button)
- ▶ IRQ signal (Interrupt ReQuest) from GPIO
- ▶ IRQ signal passed to IRQ controller
- ▶ IRQ signal passed to core



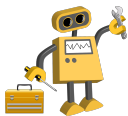
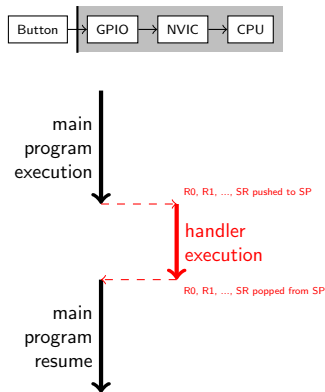
Interrupts: principles

- ▶ main program executes (main loop)
- ▶ change in the input (GPIO button)
- ▶ IRQ signal (Interrupt ReQuest) from GPIO
- ▶ IRQ signal passed to IRQ controller
- ▶ IRQ signal passed to core
- ▶ execution stopped, context saved
- ▶ selection of IRQ handler



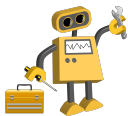
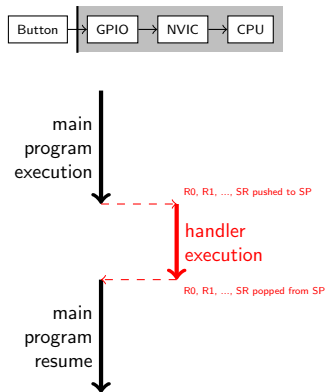
Interrupts: principles

- ▶ main program executes (main loop)
- ▶ change in the input (GPIO button)
- ▶ IRQ signal (Interrupt ReQuest) from GPIO
- ▶ IRQ signal passed to IRQ controller
- ▶ IRQ signal passed to core
- ▶ execution stopped, context saved
- ▶ selection of IRQ handler
- ▶ execution of IRQ handler



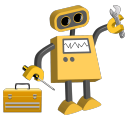
Interrupts: principles

- ▶ main program executes (main loop)
- ▶ change in the input (GPIO button)
- ▶ IRQ signal (Interrupt ReQuest) from GPIO
- ▶ IRQ signal passed to IRQ controller
- ▶ IRQ signal passed to core
- ▶ execution stopped, context saved
- ▶ selection of IRQ handler
- ▶ execution of IRQ handler
- ▶ context restored, execution resumed

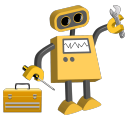
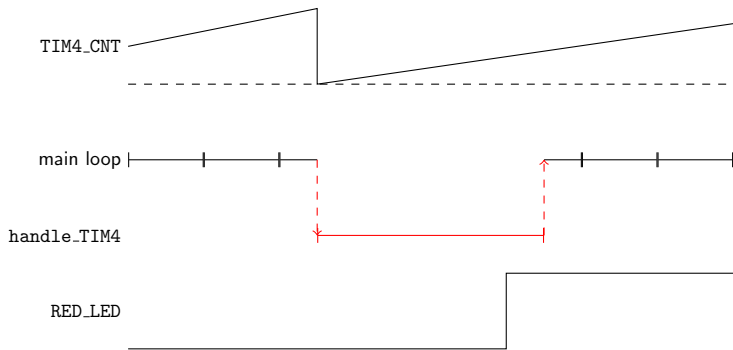


Blinking with Interrupt

```
void handle_TIM4() {  
    /* invert LED */  
    if ((GPIO_ODR & (1 << RED_LED)) == 0)  
        GPIO_BSRR = 1 << RED_LED;  
    else  
        GPIO_BSRR = 1 << (RED_LED << 16);  
  
    /* release TIM4 and IRQ */  
    TIM4_SR &= ~TIM_UIF;  
    NVIC_ICPR(TIM4_IRQ >> 5) &= 1 << (TIM4_IRQ & 0x1f);  
}  
  
int main() {  
    // initialization  
    ...  
  
    // main loop  
    while(1) {  
        // B1 test  
        ...  
  
        // B2 test  
        ...  
    }  
}
```



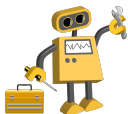
Interrupt Timetable



Cortex-M4 IRQ Vector Table

Basically mapped at address 0x0000 0000 at startup.

Address	Name	Description
0000 0000	SP_INIT	Initial stack address
0000 0004	Reset	Startup address
0000 0008	NMI	Non-Maskable Interrupt
0000 000C	Hard Fault	
0000 0010	MemManage	Memory management
0000 0014	BusFault	Memory access fault
0000 0018	UsageFault	Undefined instruction
0000 001C-0028	Reserved	
0000 002C	SVCall	System Call
0000 0030	Debug Monitor	
0000 0034	Reserved	
0000 0038	PendSV	Pending System Call
0000 003C	SysTick	System tick timer
00000 0040-0184	I/O IRQ	



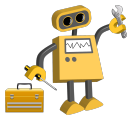
I/O IRQ: number $N \implies \text{address} = 0x40 + N \times 4$

Enabling IRQ in the Core

```
#define ENABLE_IRQS    __asm (" CPSIE _I " )
```

```
#define DISABLE_IRQS  __asm (" CPSID _I " )
```

Hint: can be used to implement critical area.



IRQ Controller

Registers:

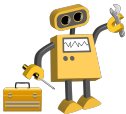
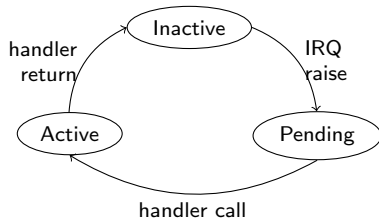
- `NVIC_ISER(0-7)` enable interrupts
- `NVIC_ICER(0-7)` disable interrupts
- `NVIC_ISPR(0-7)` set pending interrupts (software)
- `NVIC_ICPR(0-7)` clear pending interrupts
- `NVIC_IABR(0-7)` active interrupts
- `NVIC_IPR(0-59)` interrupt priorities

Accessing to the interrupt number N (32-bits registers):

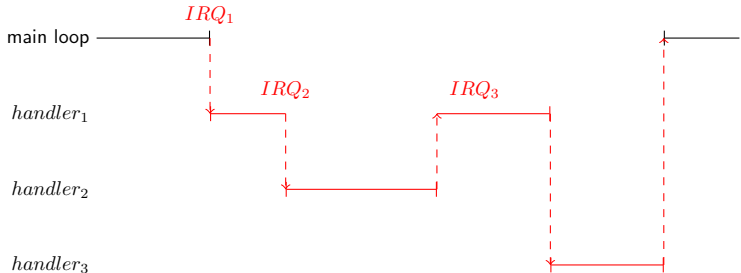
```
NVIC_XXX( $N \gg 5$ ) = 1 << ( $N \& 0x1f$ );
```

Setting the priority P (only bits 7-4 are used):

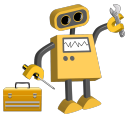
```
NVIC_IPR( $N$ ) =  $P$ ;
```



Priorities in NVIC



$$P_{handler_2} > P_{handler_1} \geq P_{handler_3}$$



Configuration of TIM4

```
DISABLE_IRQS;           // or just disable device

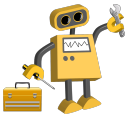
// configure NVIC
NVIC_ICER(TIM4_IRQ >> 5) = 1 << (TIM4_IRQ & 0X1f);
NVIC_IRQ(TIM4_IRQ) = (uint32_t)handle_TIM4;
NVIC_IPR(TIM4_IRQ) = 0;

// purge pending IRQ
NVIC_ICPR(TIM4_IRQ >> 5) = 1 << (TIM4_IRQ & 0X1f);

// configure TIM4
TIM4_CR1 &= ~TIM_CEN;
TIM4_PSC = WAIT_PSC;
TIM4_ARR = WAIT_DELAY;
TIM4_EGR = TIM_UG;
TIM4_SR = 0;

// enable IRQ
NVIC_ISER(TIM4_IRQ >> 5) = 1 << (TIM4_IRQ & 0X1f);
TIM4_DIER = TIM_UIE;

// starting all
ENABLE_IRQS;
TIM4_CR1 |= TIM_CEN;
```



Writing Handlers

Interrupt handler should be short

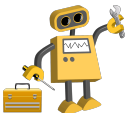
⇒ do not block other interrupts for too long!

⇒ danger of missing IRQ

How to fix big handler:

- ▶ transfer some processing to the main loop
- ▶ use global variable (boolean, etc) to trigger it from the IRQ handler

Warning: as interrupts preempt the main loop ⇒ same issues as multi-thread communication



```
volatile int TIM4_triggered = 0;
```

```
void handle_TIM4() {  
    TIM4_ARR = HALF_PERIOD;  
    TIM4_triggered = 1;  
    TIM4_SR &= ~TIM_UIF;  
}
```

```
int main() {  
    ...  
    // main loop  
    while(1) {  
  
        /* invert LED */  
        if(TIM4_triggered) {  
            TIM4_triggered = 0;  
            if((GPIO_ODR & (1 << RED_LED))  
               == 0)  
                GPIO_BSRR = 1 << RED_LED;  
            else  
                GPIO_BSRR =  
                    1 << (RED_LED << 16);  
        }  
  
        // B1 test  
        // B2 test
```

Exercise

Goal: Morse code emitter with LED light.

- ▶ dot – 100ms
- ▶ dash – 300ms
- ▶ space – 100ms
- ▶ output to GREEN_LED

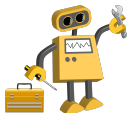
```
#define TIME_100MS    APB1_CLK/PSC/10

#define END            0
#define DOT            1
#define DASH           2

int message[] = {
    DOT, DOT, DOT,           // S
    DASH, DASH, DASH,        // O
    DOT, DOT, DOT,           // S
    END
};
```

Using the definitions on the left, write the code for:

- 1 configuring TIM4 with a period of 100ms,
- 2 writing TIM4 IRQ handler to emit the message,
- 3 configuring the TIM4 interrupt.



Debugging with Interrupts

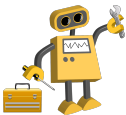
- ▶ interrupts supports break-points from the debugger
- ▶ but interrupts are caused by external devices (sensors)
- ▶ CPU is stopped (and possibly device drivers)
- ▶ but not outside world
- ▶ often: break, examine and reset

Interrupts are hard to debug: they are asynchronous!

Hints to debug interrupts:

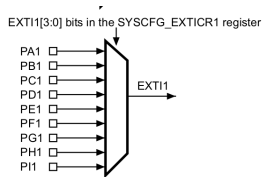
- ▶ IRQ is never raised:
check the activation of IRQ: core level, NVIC level, device level
- ▶ After first IRQ, it is re-raised even if it is not supposed to be:
in the handler, IRQ reset is not performed in the device.
- ▶ one-shot IRQ: device IRQ not reset
- ▶ You get BusFault, UsageFault:
there is a data/code error:
break-point in the handler, step-by-step until exception raise.

Final hint: the hardware registers can be displayed from the debugger!

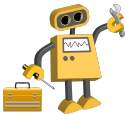


Interrupts with GPIO

In STM32F4, no support for IRQ in GPIO
 ⇒ use of EXTI (EXternal Interrupts):



At most, 16 IRQ can be generated over $11 \times 16 = 176$ pins!



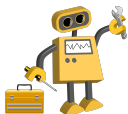
Mapping to Interrupts

Address	Name	EXTI lines	GPIO <i>X</i> pin
0000 0058	EXTI0	0	0
0000 005C	EXTI1	1	1
0000 0060	EXTI2	2	2
0000 0064	EXTI3	3	3
0000 0068	EXTI4	4	4
0000 009C	EXTI9_5	9, ..., 5	9, ..., 5
0000 00E0	EXTI15_10	15, ..., 10	15, ..., 10

EXTI9_5 and EXTI15_10 require polling!

Mapping GPIO pins $\iff$ EXTI lines must be chosen (application dependent):

- ▶ $B_0 \rightarrow \text{EXTI0}$
- ▶ $B_1 \rightarrow \text{EXTI1}$



Configuring EXTI

`EXTI_IMR` Interrupt Mask Register

`EXTI_RTSR` Rising Trigger Selection Register

`EXTI_FTSR` Falling Trigger Selection Register

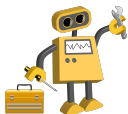
`EXTI_SWIER` SoftWare Interrupt Event Register

`EXTI_PR` Pending Register (to reset after IRQ)

`SYSCFG_EXTICRi` EXTI Configuration Register

1 bit / EXTI line

`SYSCFG_EXTICRi`: 4 bits/line, 4 line/register ($i \in [1, 4]$)



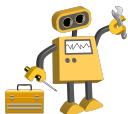
Configuring the IRQ

```
DISABLE_IRQS;
```

```
// B1 mapped to PA0/EXTI0
// configure EXTI to select GPIO A (0)
SYSCFG_EXTICR1 = REP_BITS(SYSCFG_EXTICR1, 0, 4, 0);
EXTI_RTSTR |= 1 << 0;    // rising edge
EXTI_FTSTR |= 1 << 0;    // falling edge
EXTI_IMR |= 1 << 0;      // enable
EXTI_PR |= 1 << 0;       // clear pending

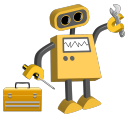
// configure NVIC
NVIC_ICER(EXTI0_IRQ >> 5) = 1 << (EXTI0_IRQ & 0X1f);
NVIC_IRQ(EXTI0_IRQ) = (uint32_t)handle_B0;
NVIC_IPR(EXTI0_IRQ) = 0;
NVIC_ICPR(EXTI0_IRQ >> 5) = 1 << (EXTI0_IRQ & 0X1f);
NVIC_ISER(EXTI0_IRQ >> 5) = 1 << (EXTI0_IRQ & 0X1f);
```

```
ENABLE_IRQS;
```



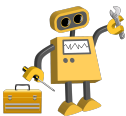
Exercise

We want to write an application that starts to blink the green LED when B1 is clicked and stops the blinking when B1 is clicked once more. To detect clicks, we will use interrupts on the GPIO.



Outline

- 1 Introduction
- 2 Requirements
- 3 Programming Input/Output
- 4 Programming with Interrupts
- 5 Conclusion



Conclusion

In this lesson,

- ▶ how to program input/output in a microcontroller,
- ▶ basics to drive GPIO and timer,
- ▶ driving by polling,
- ▶ driving by interrupts,
- ▶ principle of interaction with real world.

In the next lessons,

- ▶ how to drive sensors and actuators,
- ▶ how to manage the memory (at compilation-time and at run-time),

